

Week9

Task1:

To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>
#include <string.h>
#include <errno.h>

int main() {
    char command[100];
    pid_t pid;
    int status;
    signal(SIGTTOU, SIG_IGN);
    printf("Enter foreground command: ");
    fflush(stdout);
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = '\0';
    if (strlen(command) == 0) {
        printf("No command entered.\n");
        return 0;
    }
    pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        setpgid(0, 0);
        signal(SIGTTOU, SIG_DFL);
        execl("/bin/sh", "sh", "-c", command, NULL);
        perror("exec");
        exit(1);
    }
    if (setpgid(pid, pid) == -1 && errno != EACCES) {
        perror("setpgid");
    }
    printf("\nJob started.\n");
    printf("Job PID: %d\n", pid);
    if (tcsetpgrp(STDIN_FILENO, pid) == -1) {
        perror("tcsetpgrp");
    }
}
```

```

        perror("tcsetpgrp");
    } else {
        printf("Terminal control transferred to job.\n");
    }
    if (waitpid(pid, &status, WUNTRACED) == -1) {
        perror("waitpid");
    }
    if (tcsetpgrp(STDIN_FILENO, getpgrp()) == -1) {
        perror("tcsetpgrp");
    }
    if (WIFEXITED(status)) {
        printf("\nJob completed.\n");
        printf("Exit status: %d\n",
            WEXITSTATUS(status));
    }
    else if (WIFSIGNALED(status)) {
        printf("\nJob terminated by signal.\n");
    }
    else if (WIFSTOPPED(status)) {
        printf("\nJob stopped.\n");
        kill(pid, SIGCONT);
        printf("Job resumed.\n");
        tcsetpgrp(STDIN_FILENO, pid);
        waitpid(pid, &status, 0);
        tcsetpgrp(STDIN_FILENO, getpgrp());
        if (WIFEXITED(status)) {
            printf("Job completed after resume.\n");
        }
    }
    }
    return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./list1
Enter foreground command: pwd

Job started.
Job PID: 670
Terminal control transferred to job.
/root

Job completed.
Exit status: 0

```

Task2

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_JOBS 100
struct Job {
    int job_id;
    pid_t pid;
    char command[100];
    char status[20];
};
struct Job jobs[MAX_JOBS];
int job_count = 0;
void add_job(pid_t pid, char command[]) {
    if (job_count >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }
    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;
    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].status, "Running");
    job_count++;
}
void update_status() {
    for (int i = 0; i < job_count; i++) {
        int status;
        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );
        if (result > 0) {
            if (WIFEXITED(status)) {
                strcpy(jobs[i].status, "Completed");
            }
            else if (WIFSIGNALED(status)) {
                strcpy(jobs[i].status, "Terminated");
            }
        }
    }
}
```

```

    }
}
}

void list_active_jobs() {
    update_status();
    printf("\n");
    printf("ACTIVE JOBS\n");
    printf("%-8s %-10s %-15s %s\n",
        "Job ID",
        "PID",
        "Status",
        "Command");

    int found = 0;
    for (int i = 0; i < job_count; i++) {
        if (strcmp(jobs[i].status, "Running") == 0) {
            printf("%-8d %-10d %-15s %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].status,
                jobs[i].command);
            found = 1;
        }
    }
    if (!found) {
        printf("No active jobs.\n");
    }
}

void display_all_jobs() {
    update_status();
    printf("\n");
    printf("ALL JOBS\n");
    printf("%-8s %-10s %-15s %s\n",
        "Job ID",
        "PID",
        "Status",
        "Command");
    for (int i = 0; i < job_count; i++) {
        printf("%-8d %-10d %-15s %s\n",
            jobs[i].job_id,

```

```
        jobs[i].pid,  
        jobs[i].status,  
        jobs[i].command);  
    }  
}  
int main() {  
    int choice;  
    char command[100];  
    while (1) {  
        printf("\n===== JOB MANAGER =====\n");  
        printf("1. Start Background Job\n");  
        printf("2. List Active Jobs\n");  
        printf("3. Display All Jobs\n");  
        printf("4. Update Job Status\n");  
        printf("5. Exit\n");  
        printf("Enter choice: ");  
        scanf("%d", &choice);  
        getchar();  
        if (choice == 1) {  
            printf("Enter command: ");  
            fgets(command,  
                sizeof(command),  
                stdin);  
            command[strcspn(command, "\n")] = '\0';  
            pid_t pid = fork();  
            if (pid < 0) {  
                perror("fork");  
            }  
            else if (pid == 0) {  
                execl("/bin/sh",  
                    "sh",  
                    "-c",  
                    command,  
                    NULL);  
                perror("exec");  
                exit(1);  
            }  
            else {  
                add_job(pid, command);  
                printf("\nJob started successfully.\n");  
            }  
        }  
    }  
}
```

```

        else {
            add_job(pid, command);
            printf("\nJob started successfully.\n");
            printf("Job ID : %d\n", job_count);
            printf("PID    : %d\n", pid);
        }
    }
    else if (choice == 2) {
        list_active_jobs();
    }
    else if (choice == 3) {
        display_all_jobs();
    }
    else if (choice == 4) {
        update_status();
        printf("\nJob status updated successfully.\n");
    }
    else if (choice == 5) {
        printf("Exiting...\n");
        break;
    }
    else {
        printf("Invalid choice.\n");
    }
}
return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./control
===== JOB MANAGER =====
1. Start Background Job
2. List Active Jobs
3. Display All Jobs
4. Update Job Status
5. Exit
Enter choice: 1
Enter command: pwd

Job started successfully.
Job ID : 1
PID    : 717

===== JOB MANAGER =====
1. Start Background Job
2. List Active Jobs
3. Display All Jobs
4. Update Job Status
5. Exit
Enter choice: /root

```

Task3:

To Analyze Signal Delivery, Understand Signal Masks,
Review Process Groups, Explore Terminal Signals, Test
Signal Behavior

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>
void handler(int sig){
    printf("\nSignal %d (%s) received\n", sig, strsignal(sig));
}
int main(){
    sigset_t set, pending;
    signal(SIGINT, handler);
    signal(SIGTERM, handler);

    printf("Process ID (PID): %d\n", getpid());
    printf("Process Group ID (PGID): %d\n", getpgid());
    sigemptyset(&set);
    sigaddset(&set, SIGINT);
    if (sigprocmask(SIG_BLOCK, &set, NULL) == -1){
        perror("sigprocmask");
        return 1;
    }
    printf("\nSIGINT is now BLOCKED.\n");
    printf("Press Ctrl+C several times within 10 seconds...\n");
    sleep(10);
    sigpending(&pending);
    if (sigismember(&pending, SIGINT))
        printf("SIGINT is pending.\n");
    else
        printf("SIGINT is not pending.\n");
    printf("\nUnblocking SIGINT...\n");
    sigprocmask(SIG_UNBLOCK, &set, NULL);
    printf("Now press Ctrl+C or run:\n");
    printf("kill -TERM %d\n", getpid());
    while (1)
    {
        pause();
    }
    return 0;
}
```

Output:

```
root@DESKTOP-VMN932K:~# ./masks
Process ID (PID): 730
Process Group ID (PGID): 730

SIGINT is now BLOCKED.
Press Ctrl+C several times within 10 seconds...
SIGINT is not pending.

Unblocking SIGINT...
Now press Ctrl+C or run:
kill -TERM 730
^C
Signal 2 (Interrupt) received
```

Task4:

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
int main() {
    pid_t pid;
    int status;
    pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(1);
    }
    if (pid == 0) {
        printf("Child started. PID = %d\n", getpid());
        while (1) {
            printf("Child is running...\n");
            sleep(2);
        }
    }
    else {
        sleep(5);
        printf("\nStopping child process...\n");
        kill(pid, SIGSTOP);
        waitpid(pid, &status, WUNTRACED);
        if (WIFSTOPPED(status)) {
            printf("Job Status: STOPPED\n");
        }
        sleep(3);
        printf("\nResuming child process...\n");
        kill(pid, SIGCONT);
        printf("Job Status: RUNNING\n");
        sleep(8);
        printf("\nTerminating child process...\n");
        kill(pid, SIGTERM);
        waitpid(pid, &status, 0);
        printf("Job Status: FINISHED\n");
    }
    return 0;
}
```

Output:

```
root@DESKTOP-VMN932K:~# ./resume
Child started. PID = 802
Child is running...
Child is running...
Child is running...

Stopping child process...
Job Status: STOPPED

Resuming child process...
Job Status: RUNNING
Child is running...
Child is running...
Child is running...
Child is running...

Terminating child process...
Job Status: FINISHED
root@DESKTOP-VMN932K:~#
```